# 01 프로젝트 개요

## FreshRun이란?

FreshRun은 Gemini AI를 활용해 서울 곳곳의 새로운 달리기 코스를 자동 생성하는 iOS 앱입니다. 같은 루트를 반복하는 대신, 사용자의 거리·분위기·경유지 설정에 맞춰 매번 다른 코스를 추천합니다. 과거 기록을 학습해 절대 같은 코스를 반복하지 않습니다.

## 해결하려는 문제

- 대부분의 러너는 습관적으로 2~3개의 동일한 루트만 반복합니다.
- 기존 앱(스트라바, 나이키런클럽)은 기록은 하지만 새 코스는 제안하지 않습니다.
- 서울에는 한강, 산책로, 골목길 등 다양한 달리기 명소가 있지만 대부분 활용되지 않습니다.

## 핵심 MVP 기능

입력: 목표 거리(km) + 출발/도착 위치 + 경유지 키워드 (예: 편의점, 화장실) + 분위기 필터

출력: AI가 생성한 달리기 코스를 네이버 지도에 표시 + 예상 시간 + 코스 하이라이트

## 일정 및 마일스톤

| 날짜 | 마일스톤 |
|---|---|
| 2026. 06. 09 | 킥오프 & 기획안 작성 |
| 2026. 06 ~ 07 | 개발 스프린트 (바이브 코딩) |
| 미정 | 중간 점검: 대면 미팅 |
| 2026. 08. 01 | 프로젝트 마감 / 런칭 |

# 02 의사결정 로그

개발 시작 전에 확정한 모든 기술·제품 결정 사항입니다. 두 협업자 모두 이 내용을 기준으로 개발을 진행합니다.

| 항목 | 결정 사항 | 비고 |
|---|---|---|
| 지도 제공자 | 네이버 지도 | 한국 도보 경로 데이터 최강, 영문 문서 지원 |
| AI 모델 | Gemini (Google) | 한국어/영어 다국어 지원 우수, 합리적인 가격 |
| 백엔드 | Supabase | 무료 티어, 서버 설정 불필요, 인증+DB 일체형 |
| 경로 입력값 | 6가지 필터 전체 | 점진적 공개 UI — 고급 필터는 기본적으로 숨김 |
| 경로 유형 | 사용자 선택 | 루프 또는 단방향 — 매 러닝마다 선택 |
| 언어 | 한국어 + 영어 | 처음부터 i18n 적용 (i18next) |
| 서비스 지역 | 서울 MVP | 런칭 후 전국 및 글로벌로 확장 |
| 로그인 (MVP) | 이메일 + 비밀번호 | MVP는 이메일 인증만. 애플·카카오·구글은 앱스토어 출시 직전 단계에서 추가 예정 |
| 비회원 모드 | 없음 | 회원가입 필수 |
| 수익화 | 런칭 시 무료 | AI 코스 생성을 추후 프리미엄 기능으로 전환 (스트라바 모델) |

## 피드백 시스템 & AI 학습 루프

별점(1~5점) 방식은 러닝 후 응답률이 매우 낮습니다. 응답이 없으면 AI가 개선할 데이터가 없고, 데이터가 없으면 코스 품질이 높아지지 않습니다. FreshRun은 마찰을 최소화한 2단계 피드백으로 이 문제를 해결합니다.

| 단계 | 화면 | 내용 |
|---|---|---|
| 1 | Run Complete | 오늘 코스 어땠나요? 좋았어요 / 별로였어요 탭 1번으로 끝 — 선택 즉시 자동 저장 |
| 2a | 눌렀을 때 | 뭐가 좋았나요? (복수 선택) ☐ 조용함 ☐ 경치 좋음 ☐ 공원 많음 ☐ 달리기 편함 ☐ 안전함 |
| 2b | 눌렀을 때 | 왜 별로였나요? (복수 선택) ☐ 언덕 많음 ☐ 사람 많음 ☐ 신호등 많음 ☐ 인도 좁음 ☐ 차량 많음 ☐ 지루함 ☐ 기타 |
| 3 | AI 반영 | 태그 데이터 → Supabase runs 테이블 저장 → 다음 Gemini 프롬프트에 자동 주입 예: "사용자가 이전에 언덕 많음·차량 많음 태그한 구역은 회피할 것" |

## AI 학습 루프 설계




- 태그 → 해당 코스 특성(조용함, 공원 등)을 긍정 패턴으로 저장 → 유사 지역·유사 조건 코스 우선 추천
- 태그 → 해당 웨이포인트 구간을 회피 목록에 추가 → Gemini 프롬프트에 "이 구역 회피" 조건으로 삽입
- 무응답 → 중립 처리. 강제하지 않음 — 다음 러닝 후 자연스럽게 누적
- 누적 데이터 → 3회 이상 동일 태그 시 해당 조건을 사용자 기본 선호도에 자동 반영

## 동적 탐색 반경 시스템

한남동처럼 언덕이 많은 동네, 강남처럼 인도가 좁은 지역에서는 동네 반경 1~2km만으로는 원하는 조건의 코스를 만들 수 없습니다. FreshRun은 목표 거리와 지역 특성에 따라 탐색 반경을 자동으로 확장합니다.

| 단계 | 로직 | 예시 |
|---|---|---|
| 1. 초기 탐색 | 현재 위치 기준 반경 (목표거리 ÷ 2)km 내에서 코스 생성 시도 | 5km 요청 → 반경 2.5km 탐색 |
| 2. 지형 분석 | Gemini가 해당 구역의 언덕·인구밀도·도로폭 데이터 분석 | 한남동 → 언덕 밀도 높음 감지 |
| 3. 반경 확장 | 조건 미충족 시 반경을 +500m씩 자동 확장하며 재탐색 | 2.5km → 3km → 3.5km → 4km |
| 4. 대안 제시 | 최대 반경(8km)에서도 조건 충족 불가 시 조건 완화 옵션 제안 | "언덕 회피 조건을 완화할까요?" |

### 지역별 시나리오 예시

- 마포구 (평지): 반경 2.5km 안에서 5km 코스 즉시 생성 완료
- 한남동 (언덕 많음): 반경 자동 확장 → 한강변 경로 포함한 4km 반경까지 탐색
- 강남 (인도 좁음): 공원·산책로 우선 탐색 후 조건 충족 시까지 반경 확장

중요: 탐색 반경이 넓어져도 출발지는 항상 유저 현재 위치입니다. 반경 확장은 경유지와 경로 선택의 자유도를 높이는 것이지, 출발점을 바꾸는 게 아닙니다.

### 로그인 전략 — MVP vs 출시

MVP 개발 기간에는 이메일 + 비밀번호 인증만 구현합니다. 소셜 로그인(애플, 카카오, 구글)은 기능 자체보다 각 플랫폼 개발자 등록, 네이티브 SDK 설정, 리뷰 과정에서 시간이 많이 소요됩니다. 핵심 기능(코스 생성, 지도, 트래킹)을 먼저 완성하고, 앱스토어 출시 직전에 소셜 로그인을 일괄 추가하는 전략이 효율적입니다.

- MVP: 이메일 + 비밀번호 (Supabase 기본 제공 — 설정 5분)
- 출시 전: 애플 Sign In (iOS 소셜 로그인 제공 시 App Store 필수 정책)
- 출시 전: 카카오 로그인 (한국 사용자 편의)
- 출시 전: 구글 로그인

### 수익화 방향 (미래)

프리미엄 전환 시 스트라바 모델 참고:

- 무료: GPS 트래킹, 러닝 기록, 수동 코스
- 프리미엄: AI 코스 생성 (Gemini), 고급 필터, 경유지 추천

Gemini API 호출은 처음부터 checkUserTier() 함수로 감싸두어, 나중에 유료화할 때 설정 변경만으로 전환 가능하도록 구성합니다.

## 03 기술 스택

아래 다이어그램은 앱 전체 시스템 구조를 보여줍니다. 프론트엔드가 Supabase·Gemini·Naver Maps와 어떻게 연결되는지 한눈에 확인할 수 있습니다.



| 레이어 | 기술 | 역할 |
|---|---|---|
| 앱 프레임워크 | React Native + Expo | iOS 앱 — 바이브 코딩에 최적화 |
| 지도 (프론트) | @mj-studio/react-native-naver-map | 앱 화면 지도 렌더링 전담. 사용자에게 보이는 모든 지도는 네이버로 통일 |
| 경로 (백엔드) | T-map 도보 API (하이브리드 보조) | 도보·산책로 Polyline 추출 전용. 프론트엔 절대 노출 안 함 |
| AI / 코스 | Gemini API (Google) | 사용자 설정 기반 코스 생성 + 동적 반경 탐색 |
| 백엔드 / DB | Supabase | PostgreSQL, 인증, 실시간 기능 |
| 상태 관리 | Zustand | 경량 전역 상태 (Redux보다 단순) |
| 인증 (MVP) | Supabase Auth | 이메일 + 비밀번호만. 소셜 로그인은 출시 전 Phase에서 추가 |
| 다국어 | i18next + react-i18next | 한국어 / 영어 지원 |
| 내비게이션 | @react-navigation | 스택 + 하단 탭 내비게이션 |
| 애니메이션 | react-native-reanimated | 부드러운 UI 전환 |

### 백엔드 라우팅 API 한계 및 대응 전략

문제: 네이버 지도 Directions API는 기본적으로 자동차 전용 내비게이션입니다. 러너를 위한 도보·산책로 경로를 그대로 사용하면 부정확한 코스가 생성됩니다.

해결 전략 — 하이브리드 방식:

- 프론트엔드 (앱 화면): 무조건 네이버 지도 SDK만 사용 — 사용자에게 보이는 모든 지도는 네이버로 통일
- 백엔드 (경로 뼈대 추출): T-map 도보 API를 서버에서 호출해 도보 Polyline 좌표만 추출, 네이버 지도에 오버레이
- 장기: Phase 10에서 네이버 Roads API 도로 스냅 보정 추가로 GPS 궤적 정제

## 04 기능 로드맵 (바이브 코딩 단계)

각 단계는 Gemini를 활용한 바이브 코딩 프롬프트 하나에 대응됩니다. 순서대로 진행하세요.



| 단계 | 기능 | 목표 | 상태 |
|---|---|---|---|
| 1 | 인증 + 온보딩 | Supabase 이메일 인증, 로그인/회원가입 화면, 선호도 폼 | 예정 |
| 2 | 코스 생성 UI | 거리 슬라이더, 분위기 선택, 루프/단방향 토글, 고급 필터 아코디언, 경유지 입력 | 예정 |
| 3 | Gemini 코스 생성 | Gemini API 호출, 웨이포인트 JSON 파싱, 과거 코스 회피, 동적 탐색 반경 로직 (지역 특성 기반 자동 확장) | 예정 |
| 4 | 네이버 지도 화면 | 다크 지도에 코스 폴리라인 표시, 출발/도착 마커, 통계 바텀시트 | 예정 |
| 5 | 코스 기록 | Supabase에서 저장된 코스 불러오기, RouteCard 리스트, 검색/필터 | 예정 |
| 6 | 러닝 완료 피드백 | Run Complete 화면: / 2버튼 → 시 태그 멀티선택 (언덕 많음·사람 많음 등) → Supabase 저장 → AI 회피 로직 반영 | 예정 |
| 7 | 실시간 러닝 트래킹 | 실시간 GPS 폴리라인 (최고 정확도), 현재 페이스/거리/시간 표시 | 예정 |
| 8 | 러닝 결과 화면 | 완주 통계, 코스 일치율(%), 칼로리, 결과 공유 카드. 피드백은 Phase 6 Run Complete 화면으로 통합 | 예정 |
| 9 | 프로필 + 설정 | 총 러닝 통계, 선호도 수정, 로그아웃 | 예정 |
| 10 | 하이브리드 경로 고도화 | T-map 도보 API로 Polyline 추출 → 네이버 지도에 오버레이. 자동차 전용 Directions API 한계 극복 | 백로그 |
| 11 | 한국어 현지화 | i18next로 전체 문자열 한국어/영어 완전 분리 | 백로그 |
| 12 | 소셜 로그인 (출시 전) | 애플·카카오·구글 로그인 추가. 앱스토어 심사 전 필수 완료. 애플 로그인은 소셜 제공 시 App Store 필수 정책 | 출시 전 |

## 05 필요한 API 키

모든 키는 .env 파일에 저장하며, 절대 GitHub에 커밋하지 않습니다. 협업자와는 개인 메시지로 공유하세요.

| 서비스 | 용도 | 비용 | 발급 경로 |
|---|---|---|---|
| Gemini API | AI 코스 생성 | 약 $0.002/회 | aistudio.google.com → API 키 발급 |
| T-map API (SK텔레콤) | 도보 경로 Polyline 추출 (백엔드 전용) | 무료 티어 | tmap.skplanet.com → 개발자 등록 → API 키 |
| 네이버 지도 Client ID | 지도 렌더링 | 무료 티어 | console.ncloud.com → AI·NAVER API → 앱 등록 |
| 네이버 지도 Client Secret | 경로 + 장소 API | 무료 티어 | 위와 동일한 앱에서 발급 |
| 카카오 네이티브 키 | 카카오 소셜 로그인 (출시 전) | 무료 | developers.kakao.com → 내 애플리케이션 |
| 애플 Sign In | iOS 필수 소셜 로그인 (출시 전) | 무료 | developer.apple.com → Certificates → App ID |
| 구글 Sign In | 구글 소셜 로그인 (출시 전) | 무료 | console.cloud.google.com → OAuth 2.0 |
| Supabase URL | DB + 인증 엔드포인트 | 무료 티어 | supabase.com → 프로젝트 → 설정 → API |
| Supabase Anon Key | 클라이언트 DB 접근 | 무료 티어 | 위와 동일 |

### Supabase 테이블 (SQL 에디터에서 실행)

- preferences — 사용자 러닝 설정 (거리, 분위기, 경유지 키워드 등)
- routes — AI가 생성한 코스 기록 (웨이포인트, 거리, 피드백 태그 등)

- runs — 실제 완주 기록 (GPS 궤적, 페이스, 코스 일치율 등)

전체 SQL 쿼리는 GitHub 저장소의 docs/API_KEYS.md에 포함되어 있습니다.

## 05-B 백엔드 연결 전략

FreshRun에는 전통적인 백엔드 서버가 없습니다. Supabase가 인증·데이터베이스·실시간 기능을 한 번에 제공합니다. "프론트엔드 ↔ 백엔드 연결"은 사실상 src/services/ 폴더의 함수 호출로 끝납니다. 포트 설정, 서버 배포, API 라우팅은 MVP 단계에서 전혀 필요하지 않습니다.

### 실제로 신경 써야 할 3가지

| 항목 | 무엇이 문제인가 | 해결 방법 | 시점 |
|---|---|---|---|
| 1. 로그인 상태 관리 | 앱이 지금 누가 로그인했는지 어떻게 아는가? | 앱 시작 시 supabase.auth.getSession() 한 번 호출. 이후 모든 DB 쿼리는 Row Level Security(RLS)가 자동으로 사용자 구분. user_id를 수동으로 전달할 필요 없음. | Phase 1 |
| 2. Gemini API 키 보안 | 클라이언트에서 직접 호출하면 API 키가 앱 번들에 노출될 수 있음 | MVP는 클라이언트 직접 호출로 시작 (개발 속도 우선). 출시 전 Supabase Edge Function으로 이전 — 서버리스 함수 파일 하나, 20줄로 해결. 키는 서버 환경변수에만 존재. | Phase 3 → 출시 전 이전 |
| 3. T-map 하이브리드 라우팅 | T-map API 키는 서버사이드에서만 호출해야 하므로 클라이언트에서 직접 부르면 안 됨 | Supabase Edge Function 하나로 처리. 앱은 Edge Function만 호출 → Edge Function이 T-map 호출 후 Polyline 좌표만 반환. | Phase 10 |

### 화면별 Supabase 연결 구조

모든 DB 연결은 src/services/supabaseService.ts 파일 하나에서 관리합니다. 각 화면은 이 파일의 함수를 import해서 호출하면 끝입니다. 서버, 포트, 배포 없음.

| 화면 | 연결 대상 | 코드 예시 |
|---|---|---|
| 로그인 화면 | Supabase Auth | supabase.auth.signInWithPassword({ email, password }) |
| 코스 생성 | Gemini API + routes 테이블 | geminiService.generateRoute() → supabase.from("routes").insert(route) |
| 지도 화면 | Naver Maps SDK | NaverMapView 컴포넌트 렌더링 (SDK 내장, 별도 API 호출 없음) |
| 실시간 트래킹 | runs 테이블 | supabase.from("runs").insert(runData) — 러닝 완료 시점에 한 번 저장 |
| 런 완료 피드백 | runs 테이블 | supabase.from("runs").update({ feedback_tags: [...] }).eq("id", runId) |
| 기록 화면 | routes 테이블 | supabase.from("routes").select("*").order("created_at", { ascending: false }) |
| 프로필 화면 | preferences 테이블 | supabase.from("preferences").upsert({ user_id, ...prefs }) |

### Row Level Security (RLS) — 반드시 Phase 1에서 설정

RLS는 Supabase의 핵심 보안 기능입니다. 활성화하면 사용자는 자신의 데이터만 읽고 쓸 수 있습니다. 코드에서 user_id 필터를 직접 작성하지 않아도 Supabase가 자동으로 처리합니다. RLS 없이 배포하면 모든 사용자의 러닝 기록이 서로 노출될 수 있습니다.

- 설정: Supabase 대시보드 → Table Editor → 테이블 선택 → RLS 활성화 → Policy 추가
- 정책 예시: CREATE POLICY "own rows" ON routes USING (auth.uid() = user_id)

- docs/API_KEYS.md의 SQL 스크립트에 이미 포함되어 있음 — 복붙으로 끝

### MVP 이후 백엔드 고도화 로드맵

| 시점 | 작업 | 이유 | 난이도 |
|---|---|---|---|
| Phase 3 | Gemini 호출 → Supabase Edge Function 이전 | API 키 보안 강화 + 프롬프트 로직 서버 관리 | 낮음 (20줄) |
| Phase 10 | T-map 라우팅 → Edge Function 처리 | T-map API 키 클라이언트 노출 방지 | 낮음 (30줄) |
| 출시 전 | Supabase 플랜 업그레이드 검토 | 무료 티어: DB 500MB, 파일 2GB 제한 | 설정만 |

## 06 UI 흐름

### 코스 생성 화면 — 점진적 공개 (Progressive Disclosure)

목표: 일반 러너가 3번의 탭으로 코스를 생성할 수 있어야 합니다. 고급 옵션은 접혀 있어 기본 화면이 깔끔하게 유지됩니다.

| 순서 | 기본 화면 (항상 표시) | 고급 필터 (탭하면 펼침) |
|---|---|---|
| 1 | 거리 설정 ( / + 버튼, 1~20km) | — |
| 2 | 출발 위치 (GPS 자동 또는 수동 입력) | — |
| 3 | 경로 유형: 루프 / 단방향 토글 | — |
| 4 | 분위기: 경치 · 도심 · 자연 · 언덕 · 평지 | 난이도: 쉬움 / 보통 / 어려움 |
| 5 | — | 언덕 회피 토글 |
| 6 | — | 혼잡한 도로 회피 토글 |
| 7 | — | 공원/녹지 선호 토글 |
| 8 | — | 경유지 키워드 (예: 편의점, 화장실) |
| 9 | 코스 생성 버튼 ↗ | — |

### 내비게이션 구조

- 하단 탭 (4개): 홈 · 코스 생성 · 기록 · 프로필
- 스택 화면: 온보딩 → 메인 탭 → 지도 화면 → 코스 상세 → 러닝 중 → 러닝 결과
- 모달: 고급 필터 · 결과 공유 카드

## 07 퀵 링크

| 리소스 | 링크 / 위치 |
|---|---|
| GitHub 저장소 | github.com/[your-username]/RunAI |
| Figma 디자인 | figma.com/ (링크 추가 예정) |
| Discord | [링크 추가 예정] |
| 네이버 지도 SDK 문서 | navermaps.github.io/ios-map-sdk/guide-eng/ |
| T-map API 문서 | tmap.skplanet.com/apis |

| 리소스 | 링크 / 위치 |
| --- | --- |
| Gemini API 문서 | ai.google.dev/gemini-api/docs |
| Supabase 문서 | supabase.com/docs |
| 카카오 로그인 문서 | developers.kakao.com/docs/latest/ko/kakaologin/rest-api |
| Expo 문서 | docs.expo.dev |

FreshRun 기획안 · v1.0 · 2026년 6월 · 서울, 대한민국 · 대외비